Homework Assignment: Sentiment Analysis and TF-IDF

Background

In Week 03, we used **sentiment dictionaries** to classify words as positive or negative within windows around trade-related terms, and then counted how often those words appeared in each text. This allowed us to compute a *net sentiment score* based on raw word counts.

In Week 04, we learned **TF-IDF (term frequency–inverse document frequency)**, which changes how words "count" by giving more weight to words that are distinctive to a document and less weight to words that are common across documents. In this assignment, you will combine two methods you have learned! The goal is to learn what happens to sentiment analysis when we weight words by TF-IDF instead of simply counting them equally (each approach will be useful in different contexts).

You will compute sentiment twice for the same texts:

1. Using raw word counts
2. Using TF-IDF–weighted words

You will then interpret what changes, why it changes, and what that tells us about how computational methods shape evidence. In particular, I am going to ask you to focus on understanding what sentiment dictionaries can and cannot tell us about a corpus, and how TF-IDF reweights text data.

- **An Important Point:** Use the same preprocessing decisions for **the entire assignment**. Create one cleaned set of tokens for all three documents (*Circle of Commerce*, *Free Trade*, and A06785), and use those tokens for both the raw-count (part I) and TF-IDF analyses (part 2). This ensures that the difference you observe is caused by TF-IDF weighting rather than different preprocessing choices

Note: I am still including coding hints. This should support those of you who are still learning R. If you don't need them—don't look at them and see if you use a different approach!

How to Submit:

Post your code on your GitHub. On **Canvas**: post your response to the interpretive questions in the Discussion section (Week 3 & 4: Sentiment Analysis and TF-IDF). Make sure to include a link to the code **AND** upload a **.csv file containing your final sentiment comparison table** (see below in section III, page 4).

Hopefully everyone is now comfortable with the submission format.

Your Task

You will compute sentiment in **two ways** for the same two texts:

1. Using **raw word counts** (Week 03 method)
2. Using **TF-IDF–weighted words** (Week 04 method)

You will then compare the results and interpret what changed. Like in homework one, we will do this step-by-step and then I will ask you to interpret your results.

Use **exactly** the same two texts as in Weeks 02–04:

- *Circle of Commerce* (Text A)
- *Free Trade* (Text B)

Use the same file paths and naming conventions introduced in the tutorials.

We will break this down into steps:

I. Raw-Count Sentiment Analysis (Week 03 Method)

Note: in Week 03 we did *targeted* sentiment analysis: we built ±30-word windows around the trade terms and scored sentiment only *inside* those windows. Here we are deliberately simplifying and scoring the *whole* document instead. I am doing this so that the raw-count and the TF-IDF results sit on the same footing (TF-IDF is a document-level weighting) and the comparison in Part III is clean.

Step 1: Tokenize and clean the text

Create a tidy table with:

- one row per word token
- lowercase words
- stopwords removed
- Normalize the "long s" (ſ) to s, as we did in Week 03

This step should look very similar to what you did in Weeks 02.

Step 2: Join tokens to the Bing sentiment dictionary

Use the **Bing** sentiment lexicon to label words as positive or negative.

R & Tidy Hint

- get_sentiments("bing") returns a table with two columns: word and sentiment.
- Use inner_join() so that only words that appear in the dictionary are kept.

Step 3: Compute raw sentiment totals

Create a summary table with **one row per document** that includes:

- number of positive words
- number of negative words
- net sentiment (positive − negative)

R & Tidy Hint

- If you have *many rows per document*, you are not done yet.
- group_by(doc_title) + summarise() collapses many rows into one.
- If you see NA values, you may need to replace them with 0.

II. TF-IDF–Weighted Sentiment Analysis (see Week 04)

Two things before you start:

- First, IDF is only meaningful *relative to a corpus*. With only two texts, a word is either in one document or in both, so IDF collapses to essentially two values and the weighting tells you very little. So compute TF-IDF over the **same three texts as in Week 04**—*Circle of Commerce*, *Free Trade*, and the "mystery" third text A06785 (Malynes)—and *then* keep only Texts A and B for the sentiment comparison. The third text is there to give IDF more information to work with, **but** three documents are obviously still a very small corpus and you don't want to take the results too seriously!
- Second, an **interpretive issue**: A TF-IDF-weighted sentiment score should **not** be interpreted simply as "how positive" or "how negative" a document is. It tells us how much TF-IDF weight is carried by words that the Bing dictionary labels positive or negative.

Step 4: Compute TF-IDF for words in each document

Important: Use the **same** preprocessing decisions for both analyses. The R objects will be different because Part I uses tidy data and Part II uses a quanteda DFM (to match the tutorials), but the underlying text-cleaning decisions should be the same.

Then compute TF-IDF weights using dfm_tfidf(), as we did in Week 04

R & quanteda Hints

- Follow the DFM → TF-IDF workflow from the Week 04 tutorial.
- Your DFM should contain all three documents when you calculate TF-IDF.
- Do **not** filter to sentiment words before calculating TF-IDF. TF-IDF needs to be calculated using the vocabulary of the corpus first.
- Remember: the third document is needed to calculate the TF-IDF weights, even though we are ultimately interested in comparing only Text A and Text B.

Step 5: Convert the TF-IDF results to a tidy table

The result of dfm_tfidf() is still a quanteda document-feature matrix. To join the words to the Bing dictionary, we need to turn that matrix back into a regular data frame and then into a tidy format with **one document-word combination per row**.

Your resulting table should contain at least:

- the document name
- the word
- its TF-IDF weight

Remove rows where the TF-IDF weight is 0.

R & Tidy Hint

- convert(your_tfidf_dfm, to = "data.frame") will turn a quanteda DFM into a regular data frame. **Note:** in Week 04, we converted the DFM to a matrix using as.matrix(). Here, we want to convert it to a data frame instead so that we can return to the tidy workflow. convert(your_tfidf_dfm, to = "data.frame") will do this. You can find more information about this **here**.
- The resulting data frame will be wide: documents are rows and words are columns.
- Use pivot_longer() to turn it into a tidy table with columns for the document, word, and TF-IDF weight.
- At this point, keep only Text A and Text B. We needed the third text to calculate IDF, but we are not analyzing its sentiment.

Step 6: Keep only sentiment-bearing words

Join your tidy TF-IDF table to the Bing sentiment dictionary.

Once this is done, inspect the sentiment-bearing words with the highest TF-IDF weights in each document. You will need these results to answer Question 3 in Part IV.

R & Tidy Hint

- Use get_sentiments("bing") and join by word.
- This join happens **after** TF-IDF has been computed.
- You are filtering the TF-IDF results, not recalculating TF-IDF using only sentiment words.
- If the table becomes much smaller, that's normal—sentiment dictionaries contain only a subset of the words in your documents.
- Try arranging the results by TF-IDF weight so that you can see which sentiment-bearing words receive the most weight.

Step 7: Compute TF-IDF–weighted sentiment totals

Create a summary table with one row per document containing:

- total TF-IDF weight of positive words
- total TF-IDF weight of negative words
- net TF-IDF sentiment

Calculate:

net TF-IDF sentiment = positive TF-IDF weight − negative TF-IDF weight

R & Tidy Hint

- You are summing **TF-IDF weights**, not counting words or rows.
- group_by() and summarise() can be used to create one row per document.
- If a document has no words in one sentiment category, you may need to replace an NA with 0. Alternatively, depending on how you organize your table, pivot_wider(values_fill = 0) can handle this for you.

III. Compare Raw vs. TF-IDF Sentiment (Last step)

Create a final table that includes:

- raw sentiment results
- TF-IDF–weighted sentiment results

Export this table as a .csv. **To be posted on Canvas.**

R & Tidy Hint

- left_join() is your friend when combining summaries.
- If your table has more than one row per document, something went wrong earlier.

IV. Answer these questions (short prose to be shared on Canvas):

1. What changed between the two methods?
2. Why did TF-IDF alter the results?
3. Which specific words drove the changes?
4. When would each method be more appropriate? What limitation does document length create for the raw-count sentiment measure?

If you want a challenge [optional and not graded]

Challenge Extension: Sentiment Concentration vs. Diffusion

We are going to ask a slightly different question. We want to know how concentrated or dispersed the sentiment-bearing distinctive vocabulary is within each document. That is, is sentiment in each text:

- driven by a few highly distinctive words, or
- spread across many moderately weighted words?

Task

Using TF-IDF–weighted sentiment words:

1. Compute total TF-IDF weight of all sentiment-bearing words in each document (positive + negative weights, without subtracting one from the other).
2. Identify the top 5 sentiment words per document.
3. Calculate the proportion of sentiment carried by those five words.

End of challenge.